

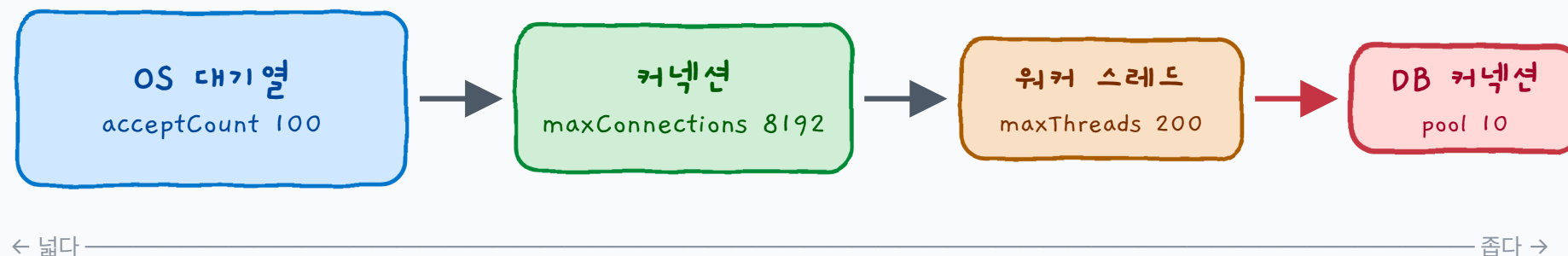
BACKEND STUDY

요청 하나가 서버를 통과하는 방법

Tomcat 스레드 풀과 DB 커넥션 풀, 두 개의 관문 이야기

00. 한 줄 결론

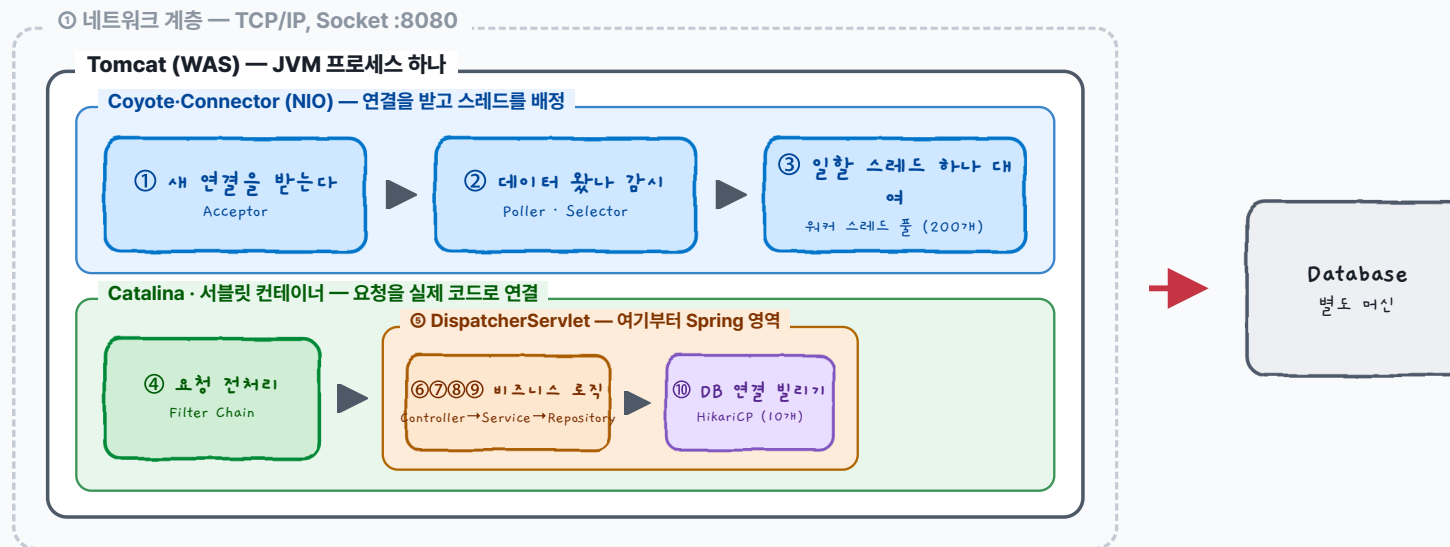
요청은 점점 좁아지는 네 개의 관문을 통과합니다



요청은 스레드 풀 → 커넥션 풀을 순서대로 통과하며,
이 중 **더 좁은 쪽**이 전체 처리량의 병목이 됩니다. 기본값 기준으로는 **커넥션 풀(10)**이 대부분의 경우 병목입니다.

01. 전체 여정 지도

네트워크 → Tomcat → 서블릿 컨테이너 → Spring → DB



핵심: 서블릿 컨테이너는 Tomcat 안의 Catalina이고, DispatcherServlet은 그 안에 등록된 서블릿 1개입니다. 그 지점부터가 Spring의 세계입니다.

01-1. 용어 정리

프로그램 · 프로세스 · 스레드

프로그램

어떤 목적을 위해 컴퓨터의 동작들을 하나로 모아 놓은 것



프로세스

현재 컴퓨터에서 **실행 중인** 프로그램



스레드

프로세스의 **작업 단위**이자 CPU 코어의 **실행 단위**

하나의 프로세스에는 여러 스레드가 할당될 수 있습니다. 그래서 프로세스 하나로도 **둘 이상의 작업을 동시에** 처리할 수 있습니다.

PART 1

왜 필요한가

스레드 풀 · NIO Connector · 커넥션 풀이 등장한 이유

02. 왜 스레드 풀인가

스레드 생성은 비쌉니다

유저 스레드 (User Thread)

1:1 매핑, 반드시 함께



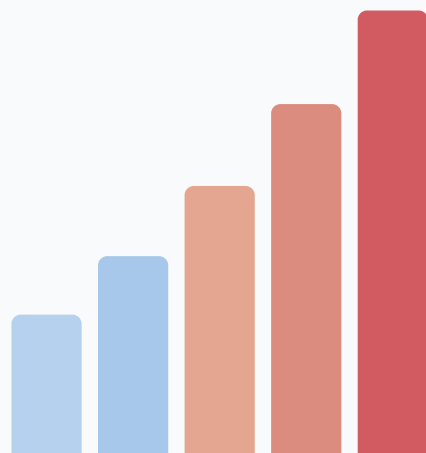
OS 커널 스레드 (Kernel Thread)

요청 도착 → `new Thread()` → 처리 → 소멸

자바는 **1:1 스레딩 모델**을 사용합니다. 유저 스레드를 만들 때마다 **OS 커널 레벨 스레드도 함께 생성**되며, 이는 커널 작업을 동반하는 비싼 연산입니다. 요청마다 이 비용이 반복되면 **응답 시간이 늘어납니다.**

02. 왜 스레드 풀인가

스레드는 무제한으로 늘어날 수 있습니다



요청 급증 →

메모리 문제

스레드가 쌓일수록 메모리 점유가 늘어납니다

CPU 오버헤드

컨텍스트 스위칭이 빈번해집니다

무제한 생성은 하드웨어에 무리를 주고, 오버헤드로 프로그램이 멈출 수도 있습니다.

02. 왜 스레드 풀인가

스레드 풀 = 재사용 + 개수 제한

문제

생성 비용이 크다

해결

미리 만들어 둔 스레드를 재사용

문제

메모리·CPU 오버헤드

해결

사용 스레드 개수를 제한

웹서버는 본질적으로 **동시 요청을 동시에 처리**해야 하는 특성을 갖습니다. 그래서 많은 웹서버가 스레드 풀을 채택합니다.

02. 왜 NIO Connector인가

빈 연결이 스레드를 붙잡지 않게

BIO — 이전 모델

연결 1개 — 스레드 1개 계속 점유

빈 연결도 스레드를 붙잡는다

연결만 열어두고 데이터를 안 보내는 클라이언트가 많아지면, 스레드가 전부 대기 중인 빈 연결에 묶여 낭비됩니다.

NIO — Tomcat 8+ 기본

Poller가 다수 연결을 Selector로 감시

데이터가 들어온 순간만 스레드 배정

연결(Connection)의 개수와 처리(Thread)의 개수를 분리해서, **만 개의 연결을 200개의 스레드로** 감당할 수 있습니다.

02. 왜 커넥션 풀인가

재사용, 그리고 DB를 지키는 문지기

① 재사용

DB 커넥션은 메모리·프로세스·인증 상태를 붙잡는 무거운 자원입니다. 비싼 생성을 매번 하지 않습니다.

② 게이트 (핵심)

풀 크기 = **DB에 동시에 던질 수 있는 쿼리 수의 상한선**. DB가 과도한 동시성으로 무너지지 않도록 보호합니다.

HikariCP의 철학 — "유저 스레드는 오직 풀 자체에서만 대기해야 하고, 커넥션 생성에서 대기하면 안 된다."
그래서 **고정 크기 풀**(`minimumIdle == maximumPoolSize`)을 권장합니다.

PART 2

어떻게 동작하는가

Acceptor · Poller · Selector와 요청 여정의 내부 구현

03. 톰캣 내부 구조

NioEndpoint 내부: Acceptor → Poller/Selector → Worker

NioEndpoint — 연결 처리 공장



Acceptor는 줄만 세웁니다. 실제로 계속 지켜보는 건 Poller와 Selector입니다.

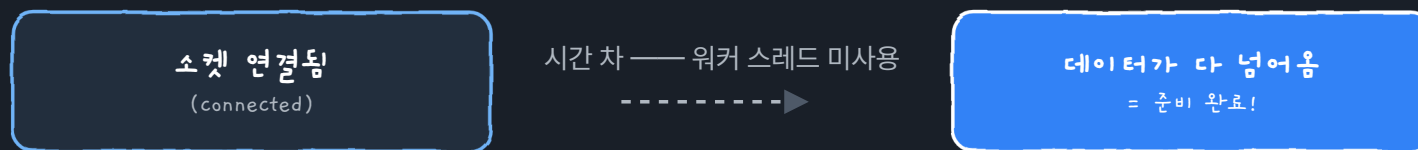
➡ 데이터가 도착한 것만

④ 워커 스레드 풀 — 실제로 일할 사람 배정

이 네 단계 덕분에, 아직 아무 말도 안 한 손님(연결만 된 소켓)은 스레드를 전혀 쓰지 않습니다. 스레드는 오직 ④ 단계, 즉 실제로 처리할 것이 생겼을 때만 배정됩니다.

03. 톱캣 내부 구조 — 핵심 개념

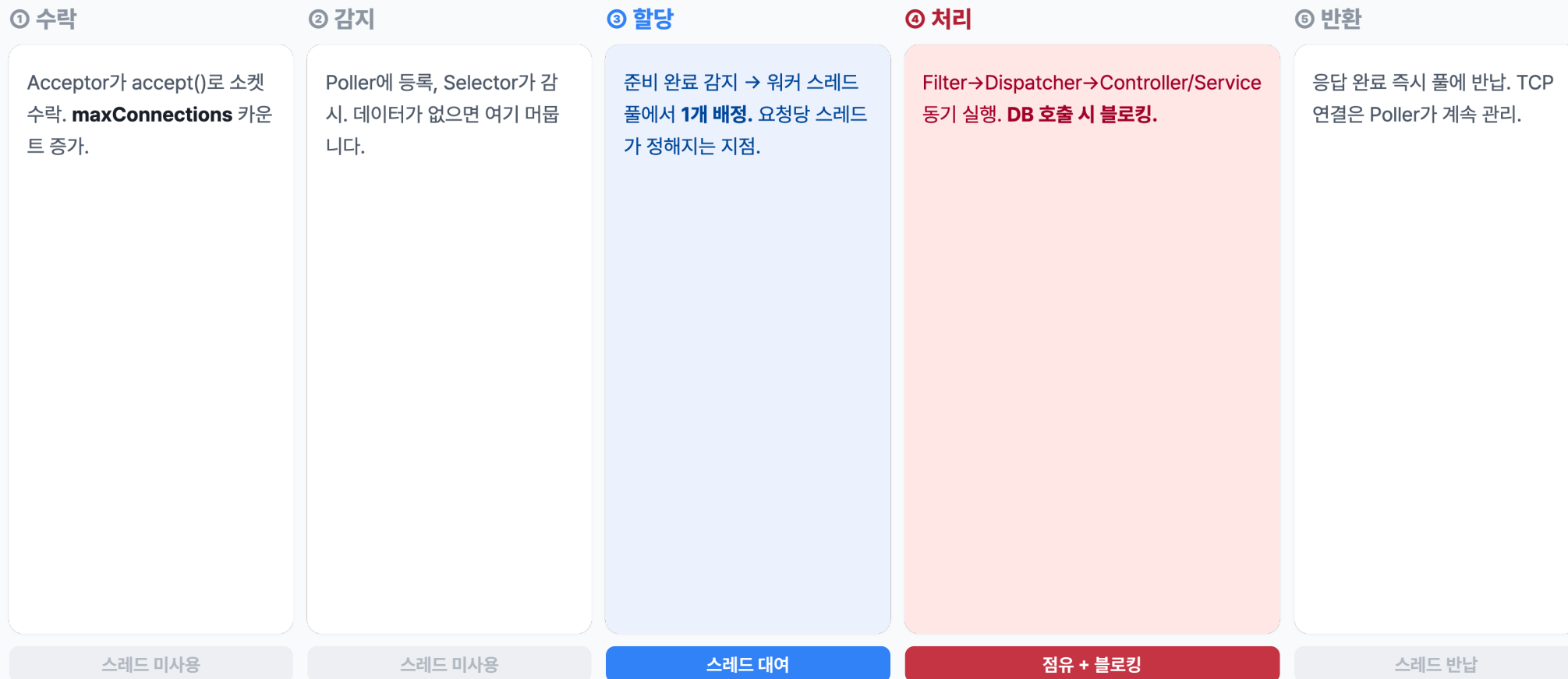
"준비된 채널" (Ready Channel)



Selector가 감지하는 것은 바로 이 "준비 완료" 상태입니다. 이것이 NIO 전체의 핵심입니다.

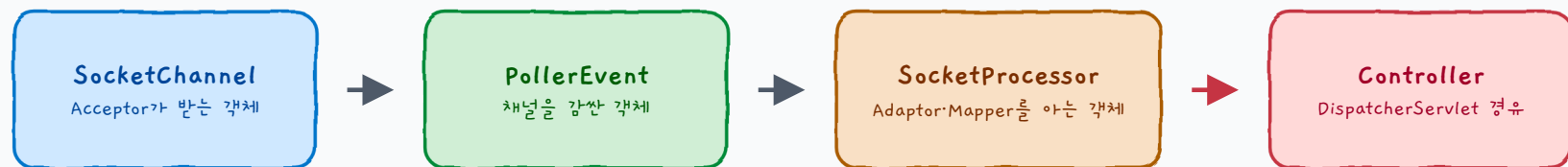
03. 스레드는 언제 할당되고, 언제 반환될까

워커 스레드의 다섯 단계



03. Worker가 Controller까지 도달하는 법

세 번의 객체 래핑



Worker는 전달받은 task를 **그냥 run만** 합니다. 서블릿 탐색과 매핑은 SocketProcessor가 아는 Adaptor와 Mapper가 처리합니다.

03. Keep-Alive

연결을 끊지 않고 재사용합니다

최초 요청

Acceptor 연결 → Poller Events 등록 → Selector 감시 → 워커 배정 → 처리

이후 요청

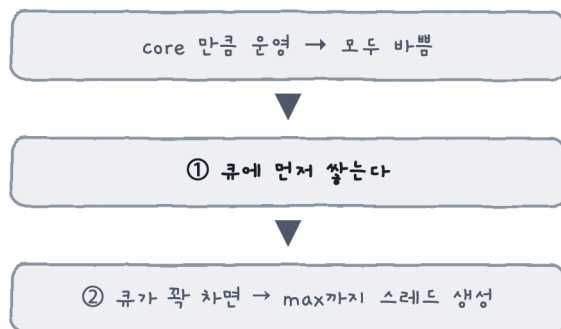
데이터 도착 → Selector 감지 → Poller → Thread Pool → 처리(Acceptor 등록 생략!)

요청 완료 이후에도 연결을 유지해서, 추가 요청에서 **핸드셰이크를 생략**합니다. TCP 연결은 Poller가 계속 관리합니다.

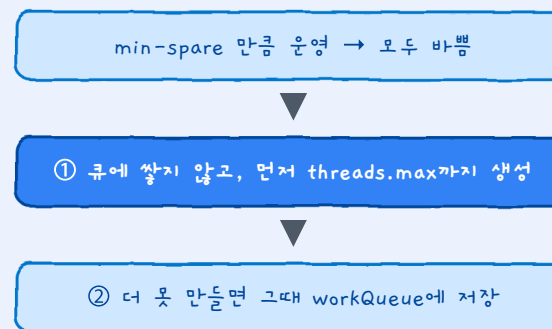
03. 흔한 오해

"바쁘면 큐부터 쌓인다"? — 톰캣은 반대입니다

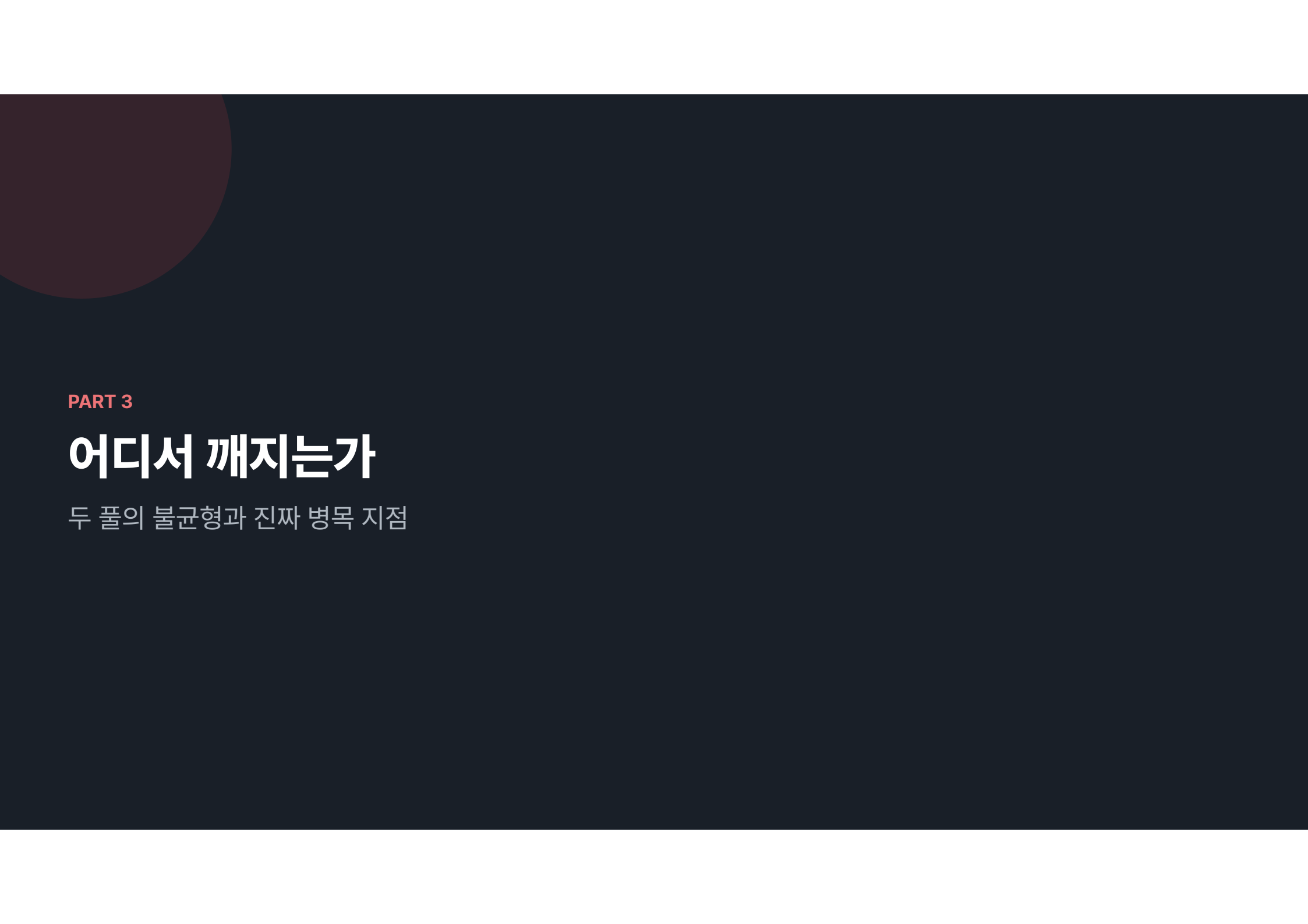
표준 ThreadPoolExecutor



Tomcat의 스레드 풀



비결: 톰캣은 **TaskQueue**를 커스터마이징해서, 아직 스레드를 더 만들 수 있으면 큐가 작업을 받지 않도록 했습니다. "큐가 거절해야 스레드를 늘린다"는 표준 규칙을 **역이용한 설계**입니다.



PART 3

어디서 깨지는가

두 풀의 불균형과 진짜 병목 지점

04. 두 풀의 관계

워커 스레드가 커넥션보다 오래 삽니다

요청 시작

요청 끝

워커 스레드 — 요청 시작부터 응답 끝까지 점유

DB 커넥션 — 쿼리 구간만 점유

@Transactional 안에서 외부 API를 호출하면 커넥션을 잡은 채로 네트워크 대기를 하게 됩니다. 커넥션 점유 시간이 늘어나고, 결국 풀 고갈로 이어집니다.

04. 두 풀의 불균형

동시 요청 200개가 들어온다면?



스레드가 부족한 게 아닙니다. **병목은 커넥션 풀**입니다. maxThreads를 400으로 늘려도 대기하는 스레드만 늘어날 뿐 아무것도 나아지지 않습니다.

04. 병목의 정의

요청은 두 개의 풀을 순서대로 통과합니다.
둘 중 **더 좁은 쪽**이 전체 처리량을 결정합니다.

실질 동시 처리량 = $\min(\text{maxThreads}, \text{maximumPoolSize})$

04. 커넥션 풀을 키우면 되지 않을까?

아닙니다. 오히려 줄이는 게 답일 때가 많습니다

커넥션 수가 **DB 서버의 CPU 코어 수**를 넘어가면, DB 안에서 **컨텍스트 스위칭**과 **락 경합**이 발생해 오히려 느려집니다.

HikariCP 벤치마크

풀 크기를 줄였더니

100ms → 2ms

스레드 풀의 교환 — 많아지면 컨텍스트 스위칭으로 느려진다

↓ 똑같은 논리가 한 번 더

커넥션 풀에서도 "많을수록 좋다"는 거짓입니다

04. HikariCP 사이징 공식

기준은 DB 서버의 자원입니다

$$\text{connections} = (\text{DB 코어 수} \times 2) + \text{디스크 스피ن들 수}$$

예시 — 4코어 + 네트워크 DB

$$(4 \times 2) + 1 =$$

9

"CPU 코어 수"는 애플리케이션 서버가 아니라 DB 서버의 코어 수입니다. 이 풀은 DB를 보호하는 게이트이므로, 기준은 DB 쪽 자원입니다. 인스턴스를 여러 대 띄운다면 총량을 인스턴스 수로 나눠 배분합니다.

05. 설정값 총정리

너무 크면, 너무 작으면

프로퍼티	기본값	관장	너무 크면	너무 작으면
threads.max	200	Worker 한계	노는 스레드↑, 메모리·CS 오버헤드	동시 처리↓ → 응답시간↑, TPS↓
max-connections	8192	Connector	—	maxThreads와 같거나 작으면 비효율
accept-count	100	Acceptor (OS)	대기열↑ → 메모리 문제	요청 몰릴 때 거절
maximumPoolSize★	10	DB 게이트	DB 컨텍스트 스위칭·락 경합 → 역효과	커넥션 대기 → 타임아웃

06. 식당에 비유하면

요리사는 200명, 화구는 10개

Acceptor

입구의 안내 직원

maxConnections

가게 안 좌석 수

acceptCount

가게 밖 대기 줄 (OS 소관)

Selector

"주문 정하셨나요?" 확인

Poller

주문서를 주방에 전달

workQueue

주방의 주문서 홀더

Worker (200)

요리사

HikariCP (10) ★

가스레인지 화구

DB

주방 설비 전체

07. 최종 요약

한 장으로 정리하는 오늘의 이야기

WHY

자바 스레드는 OS 스레드와 1:1 → 생성 비쌈, 많으면 컨텍스트 스위칭 폭증. **스레드 풀**이 재사용+개수 제한으로 해결하고, **NIO**가 연결과 처리 개수를 분리합니다.

HOW

Acceptor → Poller/Selector(**준비된 채널만**) → 워커 배열 → Filter~Repository(**블로킹**) → HikariCP → DB → 스레드 즉시 반납.

TUNE

진짜 병목은 **커넥션 풀**. maxThreads를 늘려도 소용없고, 풀은 키우기보다 **connections=(DB코어×2)+스핀들**로 줄이는 게 답. 레버는 **커넥션 점유 시간 단축**.

`acceptCount(100) → maxConnections(8192) → maxThreads(200) → HikariCP(10)` ★ 진짜 병목

한 줄 기준

**"스레드가 일을 하고 있는가,
기다리고 있는가?"**

감사합니다